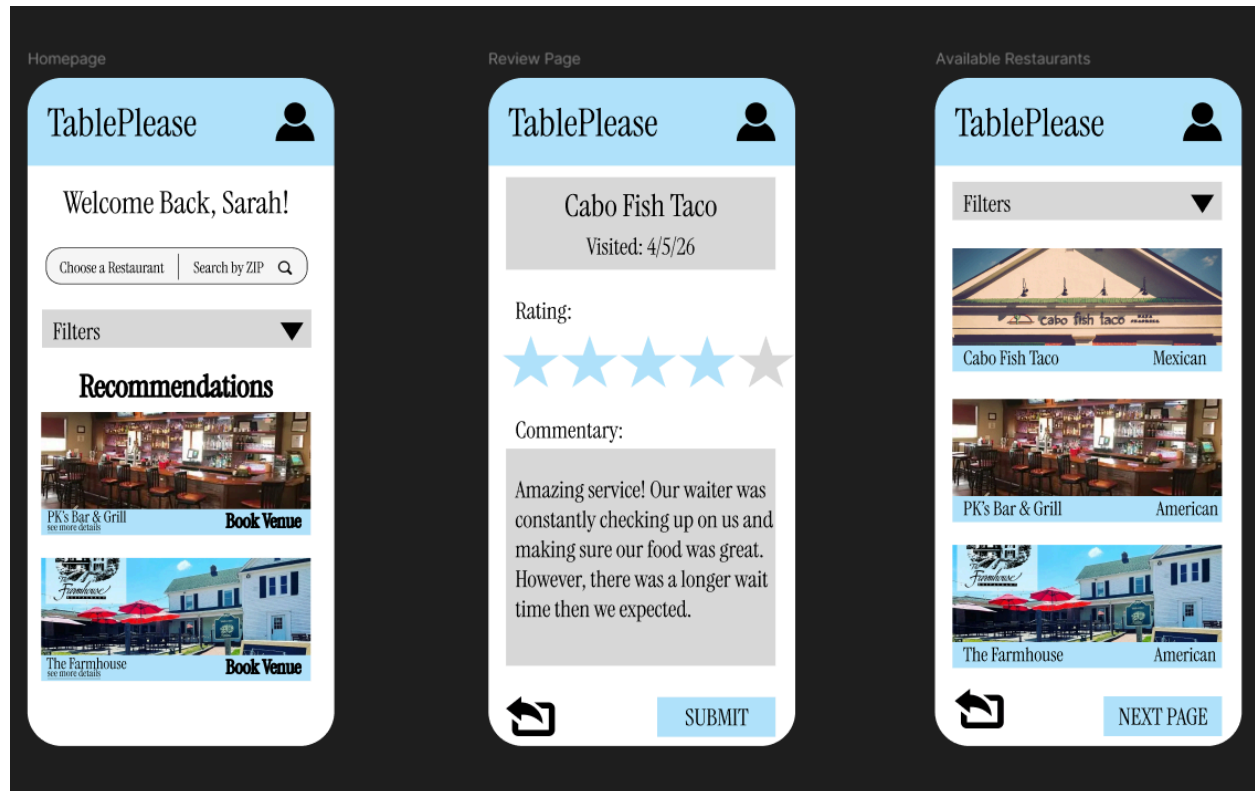


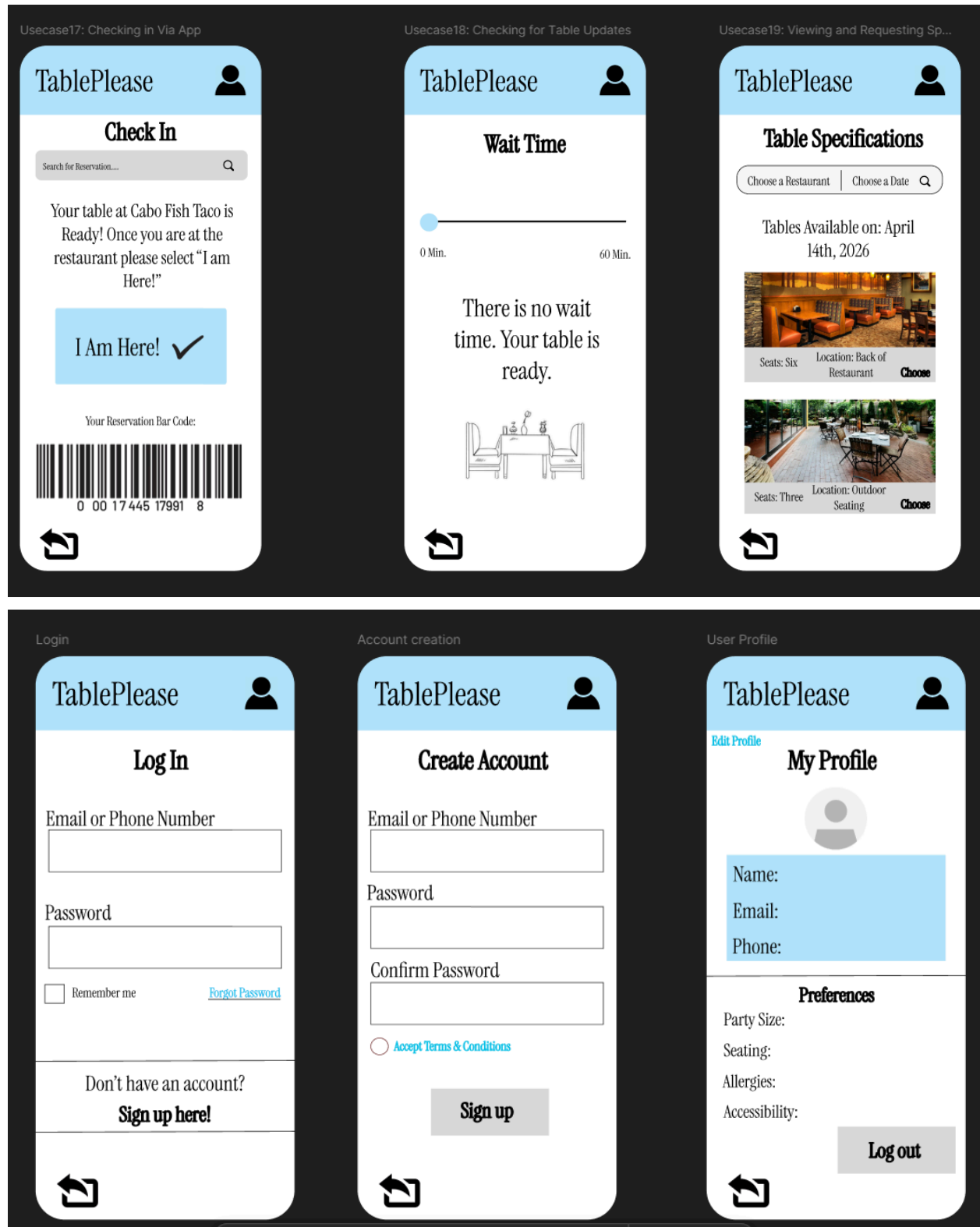
Arshiya, Parsa, Pari, Mohga, Puneetha, Neha

UI Design or Mock Uls:

Figma link:

<https://www.figma.com/design/iXUB9SefOBTAcMjNjRySo1B/USACASEUIDesgins-604?node-id=0-1&t=bstmUOIOTpkrqCW3-1>





Arshiya, Parsa, Pari, Mohga, Puneetha, Neha

Restaurant Registration

TablePlease

Register Your Restaurant

Restaurant Name

Email

Phone

Address

Password

☐ [Accept Terms & Conditions](#)

Upload Docs

Sign up

Restaurant Profile

TablePlease

Restaurant Profile

Restaurant Name

Description

Cuisine Type

American ▼

Hours

Photos

+

+

+

Save Changes

Configure Table Layout

TablePlease

Table Layout Editor

+ Table

+ Booth

+ Bar

T1
Cap: 4
Window

T2
Cap: 2
Standard

T3
Cap: 6
Booth

T4
Cap: 8
Outdoor

T5
Cap: 4
Standard

+ Add

Selected: T1 ▼

Capacity

4 ▼

Type

Window ▼

Save Changes

Usecase20:Booking the restaurant for a pri...

TablePlease

Private Event Booking

Search for Restaurant...



Cabo Fish Taco
see more details

Book Venue



PK's Bar & Grill
see more details

Book Venue



The Farmhouse
see more details

Book Venue

NEXT PAGE

Usecase2: Live bill

TablePlease

Bill

Pre-ordered Items

Spring Rolls X 10 : 10\$

Live Items

Shrimp Tacos X 2: 15\$ 5:38PM

Total: 25\$
Tax 2.5%: 15 cents
Included 10% Tip: 2.5\$
Final Total: 27.65\$

Pay Bill

usecase4: Order tracking website

TablePlease

Wait Time

0 Min. 60 Min.

Preparing Order
Est: 4:30 PM





Algorithm Design with Pseudo-Code:

Algorithm 1: UC1 - Reserve Booking with Pre-Ordering

Input: p —party size, dt —date/time, r —restaurantID, $items$ —pre-order list

Output: Confirmation status and ID

```
1.1 if  $p \leq 0$  or  $dt$  is in the past then return "Invalid Input Error"
1.2  $available \leftarrow checkTableAvailability(r, dt, p)$ 
1.3 if  $available$  is False then
1.4     return "No availability for selected time"
1.5 if user selects "Order Beforehand" then
1.6     foreach item in  $items$  do
1.7         validateAllergies(item, userProfile)
1.8  $savePreOrder(items)$ 
1.9  $resID \leftarrow createReservation(r, dt, p)$ 
1.10 return  $resID$ 
```

Algorithm 2: UC2 - Digital Bill Tracking and Payment

Input: $resID$ —reservation ID, $method$ —digital payment type

Output: Receipt and closing status

```
2.1  $bill \leftarrow fetchActiveCharges(resID)$ 
2.2 foreach  $charge$  in  $bill$  do
2.3      $total \leftarrow total + charge.price$ 
2.4  $status \leftarrow processDigitalPayment(total, method)$ 
2.5 if  $status$  is "Declined" then
2.6     return "Payment Failure: Please retry"
2.7  $closeBill(resID)$ 
2.8  $sendDigitalReceipt(userEmail)$ 
```

Algorithm 3: UC3 - Manage Order for Pickup or Delivery

Input: $cart$ —selected items, $type$ —"Order" or "Catering", $time$ —fulfillment time

```
3.1 if  $cart$  is empty then
3.2     return "Cart Error"
3.3 if  $type$  is "Catering" then
3.4 if  $time \leftarrow currentTime + 24hours$  then
3.5     return "Catering requires 24h notice"
3.6  $total \leftarrow calculateTotalWithFees(cart, type)$ 
3.7  $paymentSuccess \leftarrow authorizePayment(total)$ 
3.8 if  $paymentSuccess$  is True then
3.9      $pushToRestaurantDashboard(cart, type, time)$ 
3.10 else
```

3.11 **return** "Payment Failed"

Algorithm 4: UC4 - Track Order Stages

Input: *orderID*

4.1 *status* $\leftarrow$ *NULL*

4.2 **while** *status* is not "Completed" **do**

4.3 *newStatus* $\leftarrow$ *pollOrderStatus(orderID)*

4.4 **if** *newStatus* $\neq$ *status* **then**

4.5 *status* $\leftarrow$ *newStatus*

4.6 *updateUI(status)*

4.7 *triggerPushNotification(status)*

4.8 *wait(10seconds)*

Algorithm 5: UC5 - Register Restaurant Account

Input: *name* - restaurant name, *email* - business email, *phone* - phone number, *address* - street address, *password* - account password, *docs* - verification files

Output: Confirmation status and pending account ID

5.1 **if** *name* = '' **or** *email* = '' **or** *phone* = '' **or** *address* = '' **then**
 return "Error: All required fields must be filled"

5.2 **if** *password* length $\leftarrow$ 8 **then**
 return "Error: Invalid email format"

5.3 **if** *email* does not match valid email format **then**
 return "Error: Invalid email format"

5.4 *duplicate* $\leftarrow$ *checkDuplicateAccount(email)*

5.5 **if** *duplicate* is True **then**
 return "Error: An account with this email already exists"

5.6 **if** *docs* is empty **then**
 return "Error: At least one verification document is required"

5.7 **foreach** *doc* in *docs* **do**
 if *doc.size* > MAX_FILE_SIZE **then**
 return "Error: File" + *doc.name* + "exceeds size limit"
 if *doc.type* not in ["pdf", "jpg", "png"] **then**
 return "Error: Unsupported file type for " + *doc.name*

5.8 *hashedPassword* $\leftarrow$ *hashPassword(password)*

5.9 *accountID* $\leftarrow$ *createPendingAccount(name, email, phone, address, hashedPassword)*

5.10 *saveVerificationDocuments(accountID, docs)*

5.11 *recordVerificationRequest(accountID)*

5.12 *sendConfirmationEmail(email, accountID)*

5.13 **return** "Pending", *accountID*

Algorithm 6: UC6 - Manage Restaurant Profile

Input: *adminID* - authenticated admin ID, *updates* - map of field names to new values, *photos* - list of new image files

Output: Update confirmation status

```
6.1 if adminID is not authenticated then
    return "Error: Unauthorized access"
6.2 if updates is empty and photos is empty then
    return "Error: No changes submitted"
6.3 foreach field in updates do
    if field = "name" and updates[field] = "" then
        return "Error: Restaurant name cannot be empty"
    if field = "hours" and not validHoursFormat (updates[field]) then
        return "Error: Invalid hours format"
6.4 foreach photo in photos do
    If photo.type not in ["jpg", "png", "webp"] then
        return "Error: Unsupported image format"
    compressedPhoto <- resizeAndCompress (photo)
    savePhoto (adminID, compressedPhoto)
6.5 saveProfileUpdates (adminID, updates)
6.6 propagateToCustomerView (adminID)
6.7 return "Profile updated successfully"
```

Algorithm 7: UC7 - Configure Table Layout and Seating

Input: *adminID* - authenticated admin ID, *layout* - list of table objects each with {*id*, *type*, *capacity*, *position*, *accessible*}

Output: Layout save confirmation or validation error

```
7.1 if adminID is not authenticated then
    return "Error: Unauthorized access"
7.2 if layout is empty then
    return "Error: Layout must contain at least one table"
7.3 ids <- empty set
7.4 positions <- empty set
7.5 foreach table in layout do
    If table.capacity <= 0 then
```

```
    return "Error: Table" + table.id + " has invalid capacity"
If table.type not in ["Standard", "Booth", "Window", "Outdoor", "Bar"] then
    return "Error: Unknown table type for" + table.id
If table.id in ids then
    return "Error: Duplicate table ID: " + table.id
If table.position in positions then
    return "Error: Conflicting placement at " + table.position
add table.id to ids
add table.position to positions
7.6 saveTableLayout (adminID, layout)
7.7 propagateLayoutToStaffDashboards (adminID)
7.8 return "Layout saved and published to all staff"
```

Algorithm 8: UC8 - Manage Waitlist and Table Status Dashboard

Input: *staffID* - authenticated staff ID, *tableID* - selected table, *newStatus* - target status ("Available", "Occupied", or "Dirty"), *partyID* - optional party to seat

Output: Updated table status and recalculated waitlist estimates

```
8.1 if staffID is not authenticated then
    return "Error: Unauthorized access"
8.2 currentStatus <- getTableStatus (tableID)
8.3 validTransitions <- {
    "Available" -> ["Occupied"],
    "Occupied" -> ["Dirty"],
    "Dirty" -> ["Available"]
}
8.4 if newStatus not in validTransitions [currentStatus] then
    return "Error: Invalid status transitions from " + currentStatus + " to " + newStatus
8.5 updateTableStatus (tableID, newStatus)
8.6 if newStatus = "Occupied" and partyID != NULL then
    removeFromWaitlist (partyID)
    notifyParty (partyID, "Your table is ready")
8.7 waitlist <- getActiveWaitlist()
8.8 availableTables <- getTablesWithStatus ("Available")
8.9 foreach party in waitlist do
    estimatedWait <- calculateWaitTime (party.size, availableTables)
    party.estimatedWait <- estimatedWait
8.10 savedWaitlistEstimates (waitlist)
8.11 broadcastDashboardUpdate (staffID)
```


8.12 **return** “Table “ + *tableID* + “ updated to “ + *newStatus*

Algorithm 9: UC9 - Online Ordering for Event Reservations

Input: *c* - *customerID*, *e* - *eventID*, *t* - selected time, *items* - menu list, *pay* - payment info

Output: Confirmation status and reservation ID

```
9.1 if c is NULL then
    return “User not logged in”
9.2 if t is invalid or t is in the past then
    return “Invalid time selection”
9.3 event <- fetchEvent(e)
9.4 if event does not exist then
    return “Event not found”
9.5 available <- checkTimeSlot(event, t)
9.6 if available is False then
    return “Selected time slot unavailable”
9.7 total <- 0
9.8 foreach item in items do
9.9     menuItem <- fetchMenuItem(item)
9.10    if menuItem does not exist then
        return “Invalid menu item”
9.11    total <- total + menuItem.price
9.12 paymentStatus <- processPayment(pay, total)
9.13 if paymentStatus == “FAILED” then
    return “Payment failed”
9.14 resID <- createEventReservation(c, e, t, items)
9.15 updateTimeSlot(event, t)
9.16 sendConfirmation(c, resID)
```

Algorithm 10: UC10 - Page for All Available Restaurants/Cuisines

Input: N/A

Output: List of available restaurants or an error message

```
10.1 restaurants <- fetchAllRestaurants()
10.2 if restaurants is EMPTY then
10.3     return “No restaurants available”
10.4 availableList <- empty list
10.5 foreach r in restaurants do
10.6     if r.status == “OPEN” then
10.7         add r to availableList
10.8 if availableList is EMPTY then
10.9     return “All restaurants are currently closed”
```

10.10 *sort availableList by rating DESC*

10.11 *display availableList*

Algorithm 11: UC11 - Recommendation Service

Input: *c* - customerID

Output: List of recommended restaurants

```
11.1 if c is NULL then
11.2   return "User not logged in"
11.3 history <- fetchUserHistory(c)
11.4 if history is EMPTY then
11.5   popular <- fetchPopularRestaurants()
11.6   return popular
11.7 prefMap <- empty map
11.8 foreach visit in history do
11.9   cuisine <- visit.cuisine
11.10  rating <- visit.rating
11.11  if cuisine not in prefMap then
11.12    prefMap[cuisine] <- 0
11.13  prefMap[cuisine] <- prefMap[cuisine] + rating
11.14 topCuisine <- getMax(prefMap)
11.15 recs <- fetchRestaurantsByCuisine(topCuisine)
11.16 if recs is EMPTY then
11.17  recs <- fetchTrendingRestaurants()
11.18 return recs
```

Algorithm 12: UC12 - Rating Your Experience

Input: *c* - customerID, *rID* - reservationID, *rate* - rating, *com* - comment

Output: Rating submission status message

```
12.1 if c is NULL then
12.2   return "User not logged in"
12.3 reservation <- fetchReservation(rID)
12.4 if reservation does not exist or reservation.customerID != c then
12.5   return "Invalid reservation"
12.6 if rate <- 1 or rate > 5 then
12.7   return "Invalid rating value"
12.8 if length(com) > 500 then
12.9   return "Comment is too long"
```

```
12.10 confirm <- getUserConfirmation()
12.11 if confirm == "CANCEL" then
12.12   return "Rating canceled"
12.13 saveRating(c, rID, rate, com)
12.14 updateRestaurantRating(reservation.restaurantID)
12.15 return "Rating submitted successfully"
```

Algorithm 13: UC13 - Create Account & Login

Input: *id* - email or phone number, *pw* - password, *mode* - "Sign Up" or "Log In"

Output: Login status and session ID

```
13.1 if id is empty or pw is empty then return "Missing Input Error"
13.2 if mode ≠ "Sign Up" and mode ≠ "Log In" then return "Invalid Mode Error"
13.3 if mode is "Sign Up" then
13.4   foreach character in pw do
13.5     if character is invalid then return "Invalid Password Format"
13.6   if length(pw) < 8 then return "Weak Password Error"
13.7   if id format is invalid then return "Invalid Email or Phone Format"
13.8   if accountExists(id) is True then return "Account Already Exists"
13.9   hashedPw <- hashPassword(pw)
13.10  createAccount(id, hashedPW)
13.11 if user selects "Forgot Password" then
13.12  sendPasswordReset(id)
13.13 return "Password Reset Sent"
13.14 if mode is "Log In" then
13.15  if accountExists(id) is False then return "Account not Found"
13.16  valid <- verifyPassword(id, pw)
13.17  if valid is False then return "Invalid Credentials"
13.18 if two-factor authentication is enabled then
13.19  code <- generateVerificationCode()
13.20  sendVerificationCode(id, code)
13.21  enteredCode <- getEnteredCode()
13.22  if enteredCode ≠ code then return "Two-Factor authentication failed"
13.23  sessionID <- createSession(id)
13.24 return sessionID
```

Algorithm 14: UC14 - Manage Personalized Profile

Input: *userID* - customer ID, *profileData* - party size, seating, allergies, accessibility needs

Output: Profile update status

```
14.1 if userIsLoggedIn(userID) is False then return "Authentication Required"
14.2 if profileData is empty then return "No Changes Submitted"
```

```
14.3 if user selects “Cancel” then
14.4   discardChanges(userID)
14.5   return “Update Cancelled”
14.6 foreach field in profileData do
14.7   if field.name is “party size” and field.value <= 0 then return “Invalid Party Size”
14.8   if field.name is “party size” and field.value is not an integer then return “Invalid Party Size”
14.9   if field.name is “allergies” and length(field.value) > 100 then return “Allergy Input Too Long”
14.10  if field.name is “accessibility needs” and length(field.value) > 100 then return “Accessibility Input Too Long”
14.11  if field.name is “seating” and field.value is not allowed then return “Invalid Seating Preference”
14.12 saveProfileData(userID, profileData)
14.13 applyPreferencesToFutureReservations(userID)
14.14 return “Profile Updated Successfully”
```

Algorithm 15: UC15 - Automated Reservation Reminders & Cancellation

Input: *reservationID* - reservation ID, *type* - “Reminder” or “Cancellation”

Output: Notification delivery status

```
15.1 reservation <- getReservationDetails(reservationID)
15.2 if reservation does not exist then return “Reservation Not Found”
15.3 if notifications are disabled then return “Notification Skipped”
15.4 if type ≠ “Reminder” and type ≠ “Cancellation” then return “Invalid Notification Type”
15.5 if type is “Reminder” then
15.6   if current time is not within reminder window then return “Outside Reminder Window”
15.7   message <- buildReminderMessage(reservation)
15.8 if type is “Cancellation” then
15.9   message <- buildCancellationMessage(reservation)
15.10 methods <- [push, email, SMS]
15.11 foreach method in methods do
15.12   if method is enabled for customer then
15.13     status <- sendNotification(method, message)
15.14     if status is True then return “Notification Sent Successfully”
15.15 retryStatus <- retrySendNotification(message)
15.16 if retryStatus is False then return “Notification Delivery Failed”
15.17 return “Notification Send Successfully”
```

Algorithm 16: UC16 - View Reservation History & Favorite Restaurants

Input: *userID* - customer ID, *restaurantId* - restaurant ID, *action* - “View History” or “Toggle Favorite”

Output: Reservation history of favorite status update

```
16.1 if userIsLoggedIn(userID) is False then return "Authentication Required"
16.2 if action ≠ “View History” and action ≠ “Toggle Favorite” then return “Invalid Action”
16.3 reservations <- getReservationHistory(userID)
16.4 if action is “View History” then
16.5     if reservations is empty then return “No reservations Found”
16.6     foreach reservation in reservations do
16.7         displayReservation(reservation)
16.8     return “Reservation History Displayed”
16.9 if action is “Toggle Favorite” then
16.10    if restaurantID is empty then return “Invalid Restaurant ID”
16.11    if restaurantExists(restaurantID) is False then return “Restaurant Not Found”
16.12    if isFavorite(userID, restaurantID) is True then
16.13        removeFromFavorites(userID, restaurantID)
16.14        return “Restaurant Removed from Favorites”
16.15    else
16.16        addToFavorites(userID, restaurantID)
16.17        return “Restaurant Added to Favorites”
```

Algorithm 17: UC17 Checking in Via TablePlease App:

Input: *reservation_information* - the restaurants name and location, *reservation_status* - if reservation is

Output: *bar_code*, *check_in_validation*

```
17.1 if user selects “Check In” then
17.2     validate(reservation_information)
17.3 if reservation_status is False then
17.4     return(“There are no reservations listed. Please make a reservation.”)
17.5 else
17.6     else if user selects “I am here”
17.7         display(bar_code)
17.8         check_in_validation is True
17.9         return(“You are all checked in! Please show your confirmation ID/ barcode to the
17.10        host station.”)
```

Use Case UC18: Checking for Table Updates

Input: *reservation_information, reservation_status, wait_time, number_of_Individuals*

Output: *table_update_visual*

```
18.1 if user selects "Check In" then
18.2   validate(reservation_information)
18.3 if reservation_status is False then
18.4   return("There are no reservations listed. Please make a reservation.")
18.5 else
18.6   else if user selects "I am here"
18.7     display(bar_code)
18.8     check_in_validation is True
18.9     return("You are all checked in! Please show your confirmation ID/ barcode to the
18.10      host station.")
18.11 if user selects "View Wait Time" then
18.12 if wait_time equals 0 then
18.13   return("There is no wait time. Your table is ready.")
18.14 else
18.15   return("Your wait time is" + wait_time + "There are" + number_of_individuals +
18.16     "individuals ahead of you.")
18.17 return table_update_visual
```

Use Case UC19: Viewing and Requesting Specific Table Arrangements

Input: *date, restuarant_information*

Output: *table_recieved*

```
19.1 if user selects "Table Specifications" then
19.2   if date is valid and restuarant_information is valid then
19.4     display_tables(date,restuarant_information)
19.5   else
19.6     return("Date or Specified Restuarant is not valid. Please change your
19.7       selection.")
19.8 if user selects "choose" then
19.9   new_selection()
19.10 if user selects "Update Preferences" then
19.11   update_preferences(new_selection())
19.12 return table_received
```

Use Case UC20: Booking the restaurant for a private event

Input: *date, restuarant_information*

Output: venue_booked

```
20.1 if user selects search then
20.2   if restaurant_information is valid then
20.3     display_available_restuarants( restaurant_information)
20.4     if customer selects “Book Venue” then
20.5       if date is valid then
20.6         payment()
20.7         return venue_booked
20.8   else
20.9     return(“Date or Specified Restuarant is not valid. Please change your
20.10    selection.)
```

Algorithm 21: UC21 - Viewing Rewards Based on Spending

Input: userID - customer ID

Output: Reward balance and available rewards

```
21.1 if userIsLoggedIn(userID) is False then return “Authentication Required”
21.2 purchaseHistory ← fetchPurchaseHistory(userID)
21.3 if purchaseHistory is empty then
21.3   return “No rewards available yet. Continue dining to earn rewards.”
21.4 rewardRules ← fetchRewardRules(userID)
21.5 totalPoints ← 0
21.6 foreach purchase in purchaseHistory do
21.7   pointsEarned ← calculateRewardPoints(purchase.amount, rewardRules)
21.8   totalPoints ← totalPoints + pointsEarned
21.9 availableRewards ← fetchAvailableRewards(totalPoints)
21.10 displayRewardBalance(totalPoints)
21.11 displayAvailableRewards(availableRewards)
21.12 return “Rewards Displayed Successfully”
```

Algorithm 22: UC22 - Apply Loyalty Tier Benefits

Input: userID - customer ID

Output: Loyalty tier and associated benefits

```
22.1 if userIsLoggedIn(userID) is False then
22.1   return “Authentication Required”
22.2 rewardPoints ← getRewardPoints(userID)
22.3 tierRules ← fetchTierThresholds()
22.4 if rewardPoints < tierRules.Silver then
22.5   tier ← “Bronze”
22.6 else if rewardPoints >= tierRules.Silver and rewardPoints < tierRules.Gold then
```

```
22.7    tier ← “Silver”
22.8 else
22.9    tier ← “Gold”
22.10 benefits ← fetchTierBenefits(tier)
22.11 updateCustomerTier(userID, tier)
22.12 displayTierBenefits(tier, benefits)
22.13 return “Loyalty Tier Applied Successfully”
```

Algorithm 23: UC23 - Restaurant Configure Reward Point System

Input: adminID - authenticated admin ID, rewardConfig - reward settings

Output: Update confirmation status

```
23.1 if adminID is not authenticated then

    return “Error: Unauthorized access”
23.2 if rewardConfig is empty then

    return “Error: No reward configuration submitted”
23.3 if rewardConfig.pointsPerDollar <= 0 then

    return “Error: Invalid reward configuration”
23.4 if rewardConfig.discountRate < 0 then return “Error: Invalid reward configuration”
23.5 if rewardConfig.bonusType not in [“Points”, “Discount”, “Promotional Bonus”]

    then return “Error: Invalid reward configuration”
23.6 saveRewardConfiguration(adminID, rewardConfig)
23.7 validateRewardConfiguration(adminID)
23.8 propagateRewardSystemToCustomerApp(adminID)
23.9 return “Reward Configuration Saved Successfully”
```

Algorithm 24: UC24 - Search Restaurants by Name or Zip Code

Input: query - restaurant name or zip code

Output: List of available restaurants or an error message

```
24.1 if query is empty then

    return “Search Input Required”
24.2 return “Search Input Required”
24.3 results ← searchRestaurantDatabase(query)
24.4 if the results are empty then

    return “No restaurants found for this search.”
24.5 filteredResults ← empty list
24.6 for each r in results do
24.7     if r.name matches query or r.zipCode matches query then
24.8         add r to filteredResults
24.9 if filteredResults is empty then
```


Arshiya, Parsa, Pari, Mohga, Puneetha, Neha

24.10 **return** “No restaurants found for this search.”

24.11 displayRestaurantResults(filteredResults)

24.12 **return** “Search Results Displayed Successfully”

Github link: